Programa1.Normalizacion

November 7, 2026

0.1 Programa1.Normalización

Noviembre 2, 2026

0.1.1 1 Normalización

Nombre: Edgar Ulloa
Fecha: *02-11-2026*
Link Repositorio: https://github.com/eulloaitq/Fundamentos-de-Machine-Learning.git

[1]:
```
from sklearn import preprocessing
import numpy as np
```

[2]:
```
# Carga de datos.
X_train = np.array([[ 1., -4., 5.],
[ 5., 0., 2.],
[ 0., 6., -3.]])
print(X_train)
```

```
[[ 1. -4.  5.]
 [ 5.  0.  2.]
 [ 0.  6. -3.]]
```

[3]:
```
# Limpieza de datos: normalización.
normalizer = preprocessing.MinMaxScaler()
X_train_norm = normalizer.fit_transform(X_train)
```

[4]:
```
# Salida de resultados.
print(X_train_norm)
```

```
[[0.2   0.    1.   ]
 [1.    0.4   0.625]
 [0.    1.    0.   ]]
```

[5]:
```
# Ejemplo para ver que los resultados son iguales si aplicamos los métodos␣
↪"fit" y "transform" por separado
X_train_2 = np.copy(X_train)
# Crea un objeto "fitted_normalizer" llamando al método "fit"
f = normalizer.fit(X_train)
# Aplica el objeto sobre los datos "X_train_2" para obtener "X_train_norm_2"
X_train_norm_2 = f.transform(X_train)
# Comprueba si "X_train_norm" y "X_train_norm_2" son iguales
print(np.array_equiv(X_train_norm, X_train_norm_2))
```

```
True
```

[6]:
```
# Normalizar un nuevo conjunto de datos de test (SOLO HAY QUE APLICAR EL MÉTODO␣
↪"transform", NO EL "fit")
X_test = np.array([[ 1., -3., 2.],
[ 3., 0., 0.],
[ 0., 4., -1.]])
# Normalizar los datos de "X_test"
X_test_norm=f.transform(X_test)
```

[7]:
```
print(X_test_norm)
```

```
[[0.2   0.1   0.625]
 [0.6   0.4   0.375]
 [0.    0.8   0.25 ]]
```